Classical vs. Deep Learning Agent

for Autonomous Driving

CarRacing-v3 — Gymnasium Box2D

Yue Wu

April 2026

Motivation: Why Autonomous Driving?

Driving is one of the highest-stakes decision tasks humans do every day.

A self-driving system has to turn raw sensor data into safe, real-time control — tens of times per second, with no room for failure.

It's also a **canonical AI problem** because it stitches together three classically hard sub-problems:

- **Perception:** where is the road, the lane, the other agents?
- **Planning:** where should the car go next?
- **Control:** how much steering, gas, brake to get there?

CarRacing-v3 is the smallest toy version of this problem that still has all three pieces. Top-down view of a randomly generated track, 96×96 pixels in, steer/gas/brake out. Simple enough to solve in a class project — complex enough that *how* you solve it matters.

The Task: CarRacing-v3

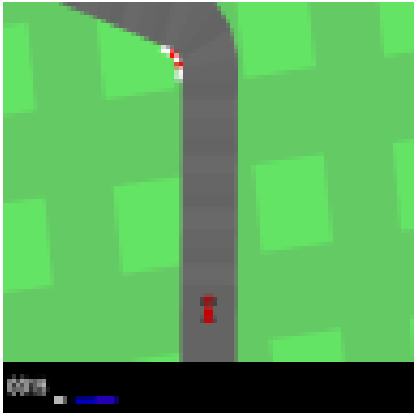
Goal: Drive a car around a randomly generated racetrack.

Input: 96×96 RGB pixel image each frame
Output: 3 continuous numbers — steer, gas, brake

Reward:

- $+1000/N$ per road tile visited (N = total tiles)
- -0.1 per frame (penalizes slow driving)
- Perfect lap $\approx 900+$

The twist: Build **two agents** that solve the same task in completely different ways, then compare.



Top-down view — car at center-bottom

Two Approaches

Classical Agent

Hand-written rules

1. Color filter → find road
2. Compute road center
3. Steer toward target

0 training time
Fully interpretable

VS

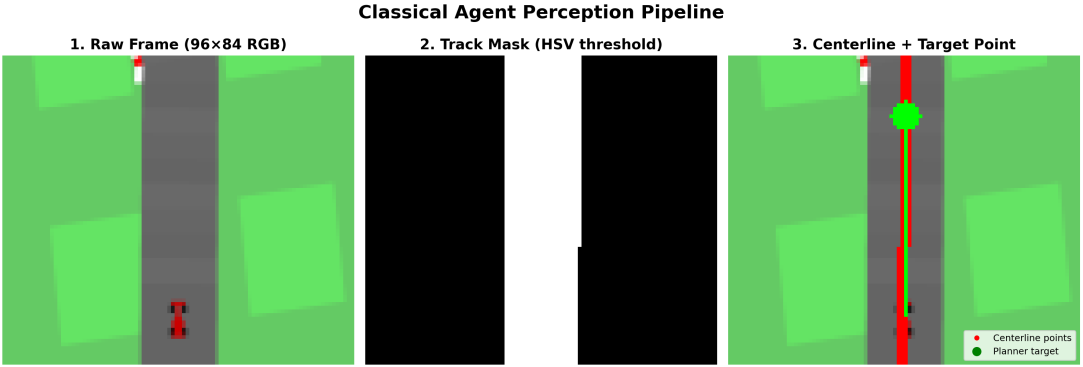
DL Agent (PPO)

Neural network learns from scratch

CNN takes raw pixels
→ outputs steer/gas/brake
Trained with PPO

6h training on GPU
Black box

Classical Agent — Perception Pipeline



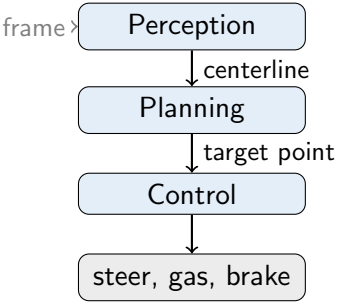
1. Convert image to HSV — gray pixels have low saturation, grass has high
2. Mark road pixels white (low color intensity = gray asphalt, or yellow stripe)
3. For each row, find the middle of the white pixels → **centerline**
4. Pick a target point ahead → steer toward it

Classical Agent — Overview

Three stages, all hand-written rules:

1. **Perception** — find the road in the image and compute its center line (red dots in Panel 3)
2. **Planning** — pick a target point up the road to aim at (green dot in Panel 3)
3. **Control** — compute how much to steer, how much gas, how much brake

Every decision is a named constant in the code — fully interpretable, zero training.



Classical Agent — Adaptive Lookahead

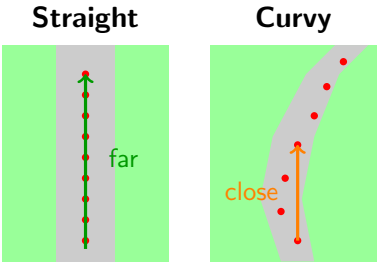
The idea: how far ahead should the car aim?

On a **straight road**, the centerline columns are all similar (low spread) → safe to aim far ahead (**55 rows**).

On a **curvy road**, the centerline columns are spread out (high spread) → aim close to react quickly (**15 rows**).

How it works:

1. Collect centerline column positions
2. Compute their spread (std deviation)
3. Low spread → far, high spread → close
4. Smooth frame-to-frame (80% old + 20% new)



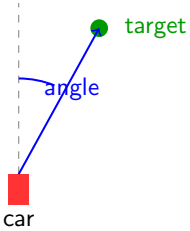
Classical Agent — Steering & Speed Control

Steering:

- Compute angle from car to target point
- Multiply by **3.4** $\rightarrow$ steering command
- Bigger angle = sharper turn

Speed control:

- Angle < 0.6 rad ($\sim 34^\circ$): gas slides $0.35 \rightarrow 0.1$
- Angle ≥ 0.6 rad: gas $= 0.1$, brake $= 0.1$
- No velocity sensor — fixed throttle schedule



$$\text{steer} = \text{angle} \times 3.4$$

$$< 0.6 \text{ rad} \rightarrow \text{gas } 0.35\text{–}0.1$$
$$\geq 0.6 \text{ rad} \rightarrow \text{gas } 0.1, \text{ brake } 0.1$$

Classical Agent — Tuning Journey

#	Change	Mean	Good≥700	Notes
0	Initial config	141	—	seed 0 only
3	First full sweep (seeds 0–9)	502	4	real baseline
5	+ adaptive lookahead + gain 3.4	602	6	
6	+ stall-trap fix	751	9	seeds 5,6 recovered
7	+ aim-only-ahead check	**786**	**9**	seed 2 fixed

Bug #1 — Stall trap: car braked to full stop on sharp turns, sat there forever. Fix: keep gas ≥ 0.1. Seed 5: 207 $\rightarrow$ 741

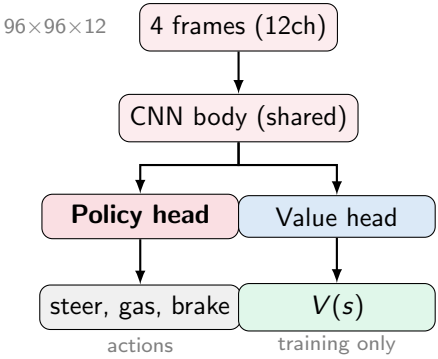
Bug #2 — T-junction: two crossing road segments $\rightarrow$ average aim lands in grass. Fix: only aim at road *ahead*. Seed 2: 120 $\rightarrow$ 470

DL Agent — Overview

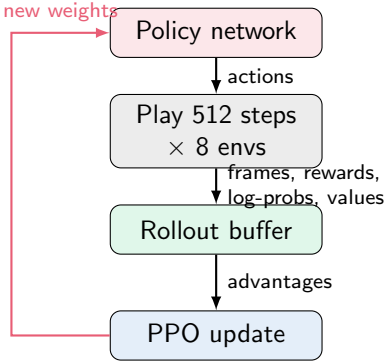
Instead of hand-written rules, a neural network learns everything from scratch by playing the game millions of times.

1. **Input** — last 4 frames stacked (12 channels)
2. **CNN body** — 3 conv layers extract visual features (shared)
3. **Policy head** — outputs steer, gas, brake
4. **Value head** — predicts future reward (training only)

No perception code, no planner, no controller — the CNN replaces *all three*.



DL Agent — PPO Training Loop



Repeat ∼4000 cycles = 2M steps

Each cycle:

1. Policy plays 512 steps on 8 parallel tracks
2. Store every (frame, action, reward, log-prob, value prediction) in a buffer
3. Compute **advantages** per step:
   actual discounted return − value prediction
   = "how much better/worse was this action than expected?"
4. Update the network weights (next slide)
5. **Discard buffer**, collect fresh data with the improved policy

DL Agent — PPO Loss Function

$$L = L_{\text{policy}} + 0.5 \cdot L_{\text{value}} - 0.01 \cdot H$$

L_{policy}

Make good actions more likely, bad actions less likely.

Clipped (range = 0.2): policy can't change too much per update. This is the "proximal" in PPO.

L_{value}

MSE between value head's prediction and actual return.

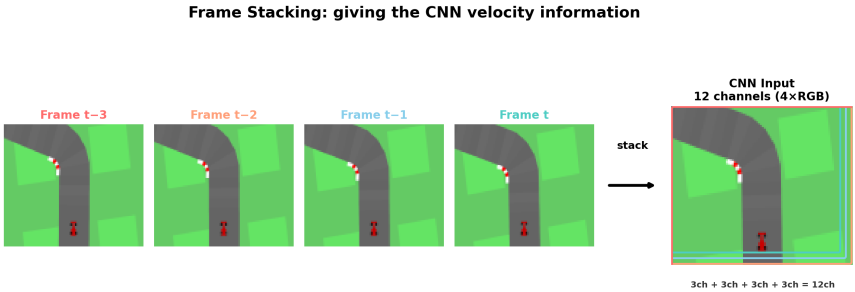
Better value predictions → better advantage estimates next cycle.

H (entropy)

Rewards the policy for staying uncertain — keeps exploring instead of committing too early to a mediocre strategy.

One combined loss, one backprop through entire network (CNN body + both heads). 10 epochs per buffer (mini-batches of 128), then discard and collect fresh data.

DL Agent — Why Frame Stacking?



A single frame is a photograph — it shows where the road is but not how fast the car moves.
By stacking 4 frames, the CNN can see pixel differences between frames → it can infer speed and turning direction.

Problem: A single frame shows *where* the road is but not *how fast* the car moves.
Fix: Stack 4 frames (12 channels). CNN sees pixel differences → infers speed and direction.

Without frame stacking: mean 402. With frame stacking: mean **855.8**, 3 full laps.

DL Agent — Training Results

Training config:

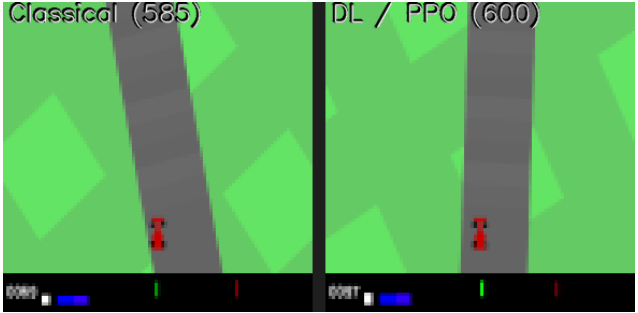
- 2M steps, 8 parallel envs
- Kaggle T4 GPU, ∼6 hours
- lr = 10^{-4}, batch = 128, 10 epochs/update
- Clip range = 0.2 (limits how much the policy changes per update)

#	Config	Mean	Laps
1	No frame stack, 500K	197	0
2	No frame stack, 2M	402	0
3	**+ frame stack, 2M**	**855.8**	**3**

What the CNN learned:

- Steer toward road center (like classical)
- **Brake before sharp turns** — something classical can't do (no velocity info)
- Recover from small mistakes by counter-steering
- Handle T-junctions without confusion (no averaging trick needed)

Demo: Side-by-Side Driving (seed 3)



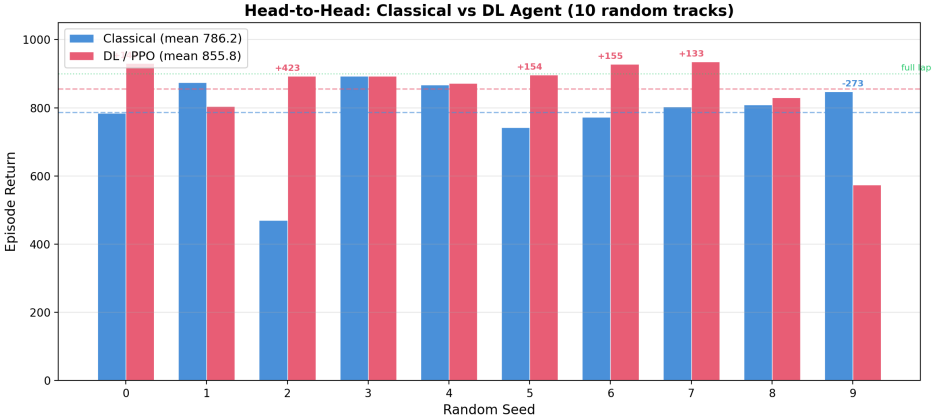
Play visuals/demo_driving.mp4 during presentation

Same track, same start. Both complete the lap.

Watch for:

- **DL** brakes before sharp turns (learned from stacked frames)
- **Classical** holds fixed throttle (no velocity sensor)
- DL recovers smoother from small deviations

Head-to-Head: Classical vs DL (10 random tracks)



DL wins: mean 855.8 vs 786.2, seed 2: +423, 3 full laps vs 0

Classical wins: seed 9: 847 vs 574

The Tradeoffs

	Classical	DL / PPO
Mean score	786.2	**855.8**
Full laps ($\geq$900)	0 / 10	**3 / 10**
Training time	**0** (hand-tuned)	$\sim$6h on T4 GPU
Engineering time	$\sim$4h of tuning	$\sim$20 min code + waiting
Interpretability	**Full** — every rule is visible	None — black box
Speed awareness	No (fixed throttle)	**Yes** (learned from frames)
Failure mode	Predictable: stalls, grass	Unpredictable: just fails
ms / frame	**4.68**	5.06

Robustness Test 1: Gaussian Pixel Noise

What is it?

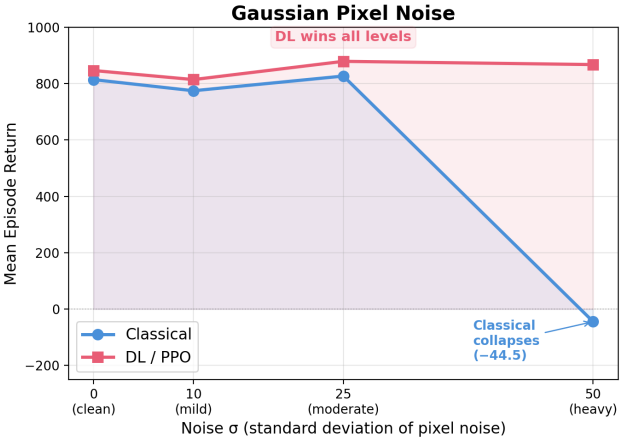
Add random static to every pixel — like a grainy camera. Each pixel gets bumped by a random amount; σ controls how strong. At σ=50, the image looks like a TV with bad reception.

Why test this?

Real cameras have sensor noise, lighting changes, dust. An agent that only works on clean images is fragile.

Question: Which agent handles noise better — the one with hard-coded color rules, or the one that learned from pixels?

Noise Results: DL Survives, Classical Collapses



Classical collapses at σ=50:

The road-finding rule uses a hard cutoff — "if color intensity $\leq$ 60, it's road." Noise bumps pixels across that boundary $\rightarrow$ road mask flickers.

DL survives:

CNN filters average over small patches — random noise cancels out within each 3$\times$3 window.

Robustness Test 2: Hue Shift (Recolored Track)

What is it?

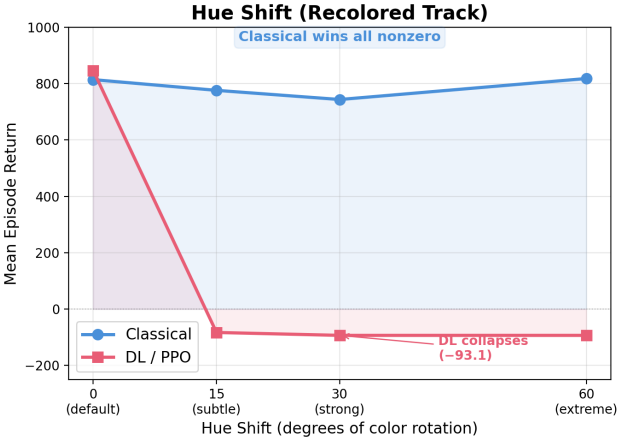
Rotate every pixel's color by a fixed amount — the gray road might become blue, the green grass might become purple. The shapes and brightness stay identical, only the colors change.

Why test this?

Different environments have different colors (desert = tan, snow = white). A robust agent shouldn't depend on memorizing one specific palette.

Question: Which agent handles recoloring better — the one that checks brightness, or the one that learned from specific colors?

Hue Shift Results: Classical Survives, DL Collapses



DL collapses at shift 15+:
The CNN memorized the exact colors from 2M training frames. It has never seen a blue road — those pixels are outside its training distribution → random actions.

Classical survives:
It checks brightness and color intensity, but ignores *which* color the road is. Rotating the hue doesn't change the brightness.

Lessons Learned

1. **DL is more adaptive.** Point it at new data $\rightarrow$ it adapts. Good when the task is complex or hard to describe in rules.
2. **Classical needs handcrafted tuning.** Feasible here (3 bugs, 4h). On a complex task — lane changes, pedestrians, weather — the rule count explodes.
3. **Classical is fully interpretable.** Every decision is a named constant. When DL fails, you get no explanation.
4. Neither is strictly better. Failure modes are complementary: DL survives noise but breaks under hue shift; classical is the opposite.
5. Training cost $\neq$ engineering cost. DL = 6h GPU + 20 min code. Classical = 0 training + 4h tuning. You pay either way.

Thank You

Code:	`github.com/.../Agents_comparison`
Task:	Gymnasium CarRacing-v3
Classical:	HSV threshold + adaptive lookahead + P-control
DL:	PPO + CnnPolicy + 4-frame stack (SB3)

Yue Wu — April 2026